

LINUX BOOT-UP ON ZYBO Z7-10 BOARD VIA SD CARD

TABLE OF CONTENTS

Introduction	2
Procedure	2
1. Hardware System Design in Vivado.....	2
2. Linux Boot File Generation Using PetaLinux.....	2
3. Boot Image Generation and SD Card Preparation	3
4. Booting Linux on the Zybo Z7-10 Board.....	3
Results	4
Conclusion	4
Knowledge Transfer	5
Appendix	6
APPENDIX A (Block Diagram)	6
APPENDIX B (PetaLinux Build Success)	7
APPENDIX C (BOOT.bin Generation)	8
APPENDIX D (Linux Successful Bootup).....	8
APPENDIX E (Entering Root Mode).....	9

Introduction

The purpose of this laboratory exercise was to build a Zynq Processing System capable of running the Linux operating system on the Zybo Z7-10 board. Unlike the previous lab, which focused on creating a custom hardware IP and interfacing it with software, this lab extended the system to support a full operating system environment. Using Vivado, a **Zynq-based hardware platform** including UART, SD card controller, timer, DDR3 memory, and the custom multiplication IP was created. The Linux kernel was then compiled using the **PetaLinux** tools and combined with the bootloader and FPGA bitstream to create a boot image. Finally, **Linux was booted from an SD card** and accessed through a serial console. This lab demonstrated the process of preparing an embedded hardware platform to run Linux and highlighted the role of bootloaders, kernel compilation, and memory systems in embedded operating systems.

Procedure

1. Hardware System Design in Vivado

1. A new directory named lab4 was created in the home folder to organize all files related to Laboratory Exercise #4.
2. Vivado was launched and a new project targeting the Zybo Z7-10 board was created.
3. A new block design was created and the ZYNQ7 Processing System (PS) IP was added.
4. Block automation was executed and the ZYBO_Z7_B2.tcl configuration file was imported to configure the processing system.
5. In the Peripheral I/O Pins tab of the PS configuration window, the required peripherals were enabled:
 - SD0 for SD card communication
 - UART1 for serial console communication
 - TTC0 timer for operating system timing support
6. The multiply IP created in Lab-3 was added to the project by copying the ip_repo directory into the Lab-4 workspace and adding it to the Vivado IP repository.
7. The multiply IP was added to the block diagram and connection automation was executed to connect the AXI interface and clock signals.
8. The block design was validated and an HDL wrapper was created.
9. The bitstream was generated and the hardware design was exported as an .xsa file including the bitstream.

The final hardware architecture of the system is shown in [APPENDIX A](#) (Block Diagram).

2. Linux Boot File Generation Using PetaLinux

10. A PetaLinux installation directory was created under /tmp for the user workspace.

```
cd /tmp
```

```
mkdir petalinux_nithingowtham25
```

11. The PetaLinux installer was copied from the shared lab directory.

```
cd petalinux_nithingowtham25  
cp /mnt/lab_files/ECEN449/petalinux-v2023.1-05012318-installer.run .
```

12. A directory was created for the PetaLinux installation and the installer was executed.

```
mkdir petalinux_install  
chmod 755 ./petalinux-v2023.1-05012318-installer.run  
./petalinux-v2023.1-05012318-installer.run --dir petalinux_install --platform "arm"
```

13. After installation completed, the PetaLinux environment variables were loaded.

```
source petalinux_install/settings.sh
```

14. A new PetaLinux project named **linux_boot_nithingowtham25** was created using the Zynq template.

```
petalinux-create -t project --template zynq -n linux_boot_nithingowtham25 --tmpdir  
/tmp/petalinux_nithingowtham25/tmp_dir
```

15. The hardware configuration of the PetaLinux project was updated using the hardware design exported from Vivado.

```
cd linux_boot_nithingowtham25  
petalinux-config --get-hw-  
description=/home/grads/n/nithingowtham25/Spring2026/ECEN749/lab4_new_linux_boot_wrapper.x  
sa --silentconfig
```

16. After successful compilation, the generated build files were verified inside the images directory.

The successful PetaLinux build process is shown in [APPENDIX B](#) (PetaLinux Build Success).

3. Boot Image Generation and SD Card Preparation

16. The Zynq boot image BOOT.bin was generated using the PetaLinux packaging command, which combines:

- First Stage Boot Loader (FSBL)
- FPGA bitstream
- U-Boot bootloader

The successful generation of the boot image is shown in [APPENDIX C](#) (BOOT.bin Generation).

17. The SD card was formatted in FAT32 format.

18. The following files were copied to the SD card:

- BOOT.bin
- image.ub
- boot.scr

4. Booting Linux on the Zybo Z7-10 Board

19. The Zybo Z7-10 board boot mode jumper JP5 was set to SD card mode.

20. The picocom serial console was launched using the command:

```
picocom -b 115200 /dev/ttyUSB1
```

21. The SD card was inserted into the Zybo board and the board was reset to begin the Linux boot process.

22. The Linux boot messages appeared on the serial console, confirming successful system initialization.

The successful Linux boot output is shown in [APPENDIX D](#) (Linux Successful Bootup).

23. After the system booted, the default user petalinux was used to log into the system.

24. The command **sudo su** was executed to switch to the root user.

Entering root mode is shown in [APPENDIX E](#) (Entering Root Mode).

(The completed Linux boot process was demonstrated to the TA.)

Results

The Linux operating system successfully booted on the Zybo Z7-10 board using the generated hardware design and boot files. After programming the FPGA bitstream and copying the required boot files (BOOT.bin, image.ub, and boot.scr) to the SD card, the board was reset and the Linux boot sequence appeared on the serial console through the picocom terminal.

The console output showed the initialization of the Zynq Processing System, loading of the Linux kernel, configuration of system peripherals, and mounting of the root file system. After the boot process was completed, the login prompt appeared on the terminal. Logging in as the **petalinux** user and executing the command **sudo su** allowed switching to the root user, confirming that the operating system was running correctly.

The successful Linux boot sequence verified that the hardware platform generated in Vivado was compatible with the Linux kernel built using PetaLinux tools. It also confirmed correct configuration of the SD card controller, DDR3 memory, timer, interrupt controller, and UART interface required for Linux operation. The boot output and successful login are shown in [APPENDIX D](#) and [APPENDIX E](#).

The completed Linux boot process was demonstrated to the TA.

Conclusion

This laboratory exercise successfully demonstrated the process of preparing a Zynq-based embedded system to run the Linux operating system. By creating a hardware platform in Vivado and compiling the Linux kernel using PetaLinux tools, the experiment illustrated the interaction between hardware configuration and operating system support in embedded systems. The ability to generate bootloaders, kernel images, and boot scripts, and successfully boot Linux from an SD card on the Zybo Z7-10 board, provided valuable experience in embedded Linux development. This knowledge is important for developing complex FPGA-based systems that require operating system support for advanced software applications.

Knowledge Transfer

- a) Compared to Lab 3, the Lab 4 microprocessor system shown in Figure 1 has 512 MB of SDRAM. However, our system still includes a small amount of local memory. What is the function of the local memory? Does this 'local memory' exist on a standard motherboard? If so, where?

Compared to Lab-3, the microprocessor system in Lab-4 includes 512 MB of DDR3 SDRAM, which is used as the **main system memory** where the Linux kernel, applications, and data reside. However, the system also includes a small amount of local memory implemented as **on-chip memory inside the Zynq Processing System (On-Chip Memory, OCM)**. The purpose of this local memory is to store critical code or data that must be accessed with very low latency during system initialization and execution. It is also useful for temporary storage and fast access operations before the external DDR memory is fully initialized.

A similar concept exists in standard computer systems. In modern processors, **cache memory (L1/L2/L3 cache)** provides fast on-chip memory that stores frequently accessed data to reduce access latency compared to main system memory (DRAM). Thus, the local memory in embedded FPGA systems serves a role similar to cache memory in traditional computing systems.

- b) After your Linux system boots, navigate through the various directories. Determine which of these directories is writable. Suppose you can create a file in one of these directories. What happens to this file when you restart the ZYBO Z7-10 board? Why? The following command lists all directories

```
$ cd /
```

```
$ ls -l
```

Note: You can test the permissions by typing 'touch <filename>' in each of the directories. If the file "<filename>" is created, that directory is writable

In our Linux system, directories such as **/proc** and **/sys** do not allow the creation of regular files because they are **virtual filesystems managed by the Linux kernel**. These directories provide system and hardware information rather than storing normal files. For example, when attempting to create a file using the command "**touch hi**" inside the **/sys** directory while operating in root mode, the system returns a "**Permission denied**" message, as shown in the attached screenshot. This indicates that these directories do not support normal file creation.

```
linux_boot_nithingowtham25:/sys# touch hi
touch: hi: Permission denied
linux_boot_nithingowtham25:/sys#
```

Although files can be created in some other directories while operating in root mode, they are **not stored permanently**. This is because the root filesystem is loaded into **RAM during the boot process from the image.ub file**. Any files created during runtime exist only in **volatile memory**, and when the Zybo Z7-10 board is restarted, the filesystem is reloaded from the original image. As a result, any files created during the previous session are lost and must be recreated if needed.

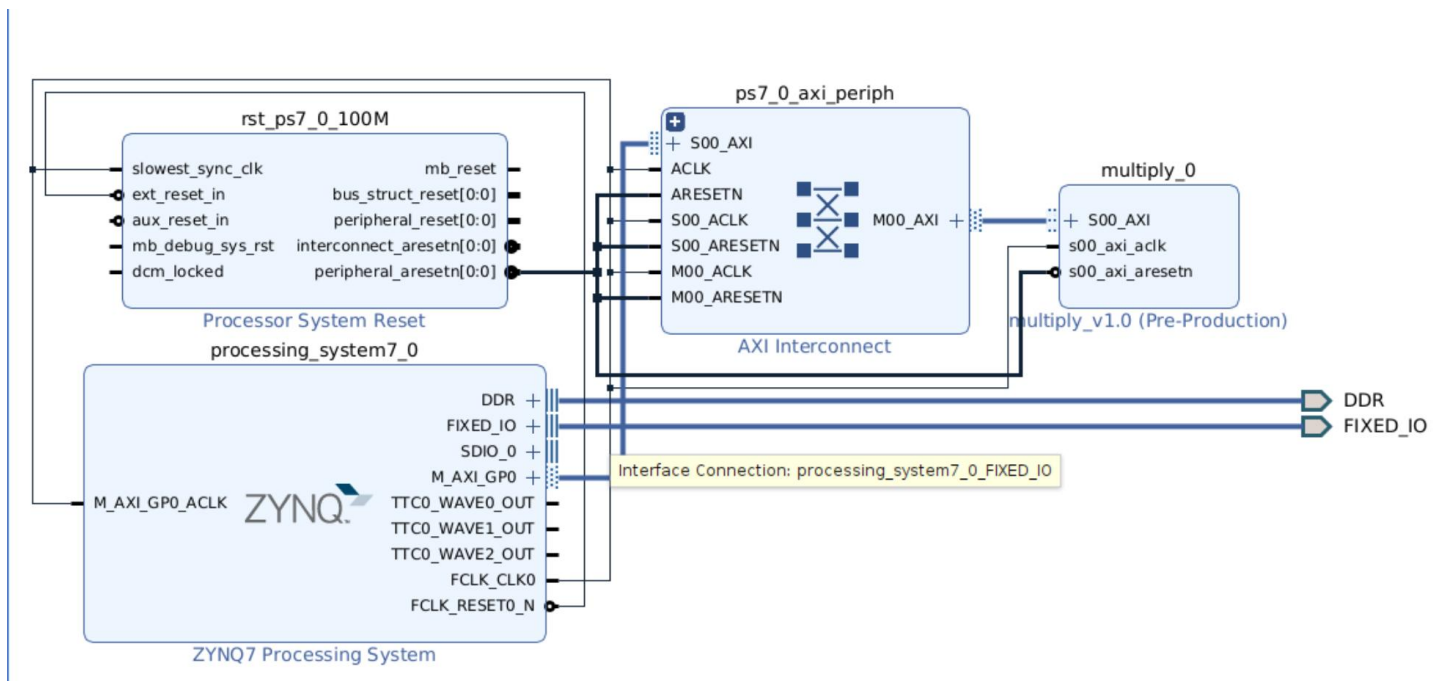
c) If you were to add another peripheral to your system after compiling the kernel, which of the above steps would you have to repeat? Why?

If a new peripheral is added after compiling the Linux kernel, several steps must be repeated. First, the hardware design in Vivado must be updated by adding the peripheral to the block diagram, regenerating the bitstream, and exporting the updated .xsa file. Next, the PetaLinux project must be reconfigured using the new hardware description so that the **device tree includes the new peripheral**. The project must then be rebuilt using petalinux-build, and a new boot image (BOOT.bin) must be generated. Finally, the updated boot files (BOOT.bin, image.ub, and boot.scr) are copied to the SD card, and Linux is booted again on the Zybo Z7-10 board using the picocom console.

These steps are required because the **Linux kernel and device tree must match the hardware configuration**. Without recompiling and updating the boot files, the operating system would not recognize or correctly communicate with the newly added peripheral.

Appendix

APPENDIX A (Block Diagram)



APPENDIX B (PetaLinux Build Success)

```

Mar 5 11:37
nithingowtham25@zach-333-em4-05:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25...
nithingowtham25@zach-333-em4-05:... x nithingowtham25@zach-333-em4-05:... x nithingowtham25@zach-333-em4-05:... x
d65c0, but the sig is locked to c2ac5f0c29da84190d0f46f4f48dcf67ef7255138a4424d0a00004f7f614a942 in SIGGEN_LOCK
EDSIGS_t-cortexa9t2hf-neon
The cracklib-native:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733
192e, but the sig is locked to 296da4145c53977745d54f468ecac65c4921499a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKE
DSIGS_t-x86-64
The cracklib:do_fetch sig is computed to be 9dc70ff8344d10501b14d2918c5da0d975de1fc55ffe7b42af17bb5c1733192e, b
ut the sig is locked to 296da4145c53977745d54f468ecac65c4921499a7abd5d76e8e45d074775f0a9 in SIGGEN_LOCKEDSIGS_t
-cortexa9t2hf-neon
The cracklib-native:do_unpack sig is computed to be 8720bf6a6cea0d038864f7fe2485c354f02f6b87091eaa065f541ce3752
22d26, but the sig is locked to c16adc0993000ecbfab876afd386d57bfd904e387a819d1c30f5b92952c7418e in SIGGEN_LOCK
EDSIGS_t-x86-64
The cracklib:do_unpack sig is computed to be 75ca96c0affc2ddb4f38787421a9ebb9c2f5b2255cf00d3568500a42c44c32db,
but the sig is locked to f19c0ff3b09b462be625586dbcd4a6481dc4f9912901fc2667a011a7a411aeba in SIGGEN_LOCKEDSIGS_
t-cortexa9t2hf-neon
The cracklib-native:do_populate_lic sig is computed to be fb8c48d2286f834773a054c9b6d1455ee3e16b98159b23a698ea9
9427a14c442, but the sig is locked to 059f83c7dcd22dda2a903132eead0fa4683ba47834aaf85c518aaadb3cda2fca in SIGGE
N_LOCKEDSIGS_t-x86-64
The cracklib:do_populate_lic sig is computed to be 968af2fd6d5dafd0adde0c99d65a5b431f0dbe2db71b85b6e98df3bddc95
462e, but the sig is locked to 6359632f75f1183510a4567460fc9ffa97ce4ad1d9b9a2052da3614dcdbd311c in SIGGEN_LOCKE
DSIGS_t-cortexa9t2hf-neon
NOTE: Executing Tasks
NOTE: Tasks Summary: Attempted 4139 tasks of which 437 didn't need to be rerun and all succeeded.

Summary: There was 1 WARNING message.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
nithingowtham25@zach-333-em4-05:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25$ cd images/linux/
nithingowtham25@zach-333-em4-05:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux$ ls -al
total 218460
drwxr-xr-x 3 nithingowtham25 nithingowtham25      4096 Mar  5 11:35 .
drwxr-xr-x 3 nithingowtham25 nithingowtham25      4096 Mar  5 11:16 ..
-rw-r--r-- 1 nithingowtham25 nithingowtham25      3016 Mar  5 11:17 boot.scr
-rw-r--r-- 1 nithingowtham25 nithingowtham25      6837 Mar  5 11:05 config
-rw-r--r-- 1 nithingowtham25 nithingowtham25    17841212 Mar  5 11:35 image.ub
drwxr-xr-x 2 nithingowtham25 nithingowtham25      4096 Mar  5 11:17 pxelinux.cfg
-rw-r--r-- 1 nithingowtham25 nithingowtham25    34567168 Mar  5 11:35 rootfs.cpio
-rw-r--r-- 1 nithingowtham25 nithingowtham25    13160092 Mar  5 11:35 rootfs.cpio.gz
-rw-r--r-- 1 nithingowtham25 nithingowtham25    13160156 Mar  5 11:35 rootfs.cpio.gz.u-boot
-rw-r--r-- 1 nithingowtham25 nithingowtham25    68882432 Mar  5 11:35 rootfs.ext4
-rw-r--r-- 1 nithingowtham25 nithingowtham25    15466496 Mar  5 11:35 rootfs.jffs2
-rw-r--r-- 1 nithingowtham25 nithingowtham25      22114 Mar  5 11:35 rootfs.manifest
-rw-r--r-- 1 nithingowtham25 nithingowtham25    23582491 Mar  5 11:35 rootfs.tar.gz
-rw-r--r-- 1 nithingowtham25 nithingowtham25    2083854 Mar  5 10:59 system.bit
-rw-r--r-- 1 nithingowtham25 nithingowtham25      18366 Mar  5 11:17 system.dtb
-rwxr-xr-x 1 nithingowtham25 nithingowtham25    1006496 Mar  5 11:20 u-boot.bin
-rw-r--r-- 1 nithingowtham25 nithingowtham25    1024862 Mar  5 11:20 u-boot-dtb.bin
-rw-r--r-- 1 nithingowtham25 nithingowtham25    1090824 Mar  5 11:20 u-boot-dtb.elf
-rwxr-xr-x 1 nithingowtham25 nithingowtham25    7840400 Mar  5 11:20 u-boot.elf
-rw-r--r-- 1 nithingowtham25 nithingowtham25     4660680 Mar  5 11:35 uImage
-rw-r--r-- 1 nithingowtham25 nithingowtham25    14132328 Mar  5 11:35 vmlinux
-rw-r--r-- 1 nithingowtham25 nithingowtham25     4660616 Mar  5 11:35 zImage
nithingowtham25@zach-333-em4-05:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux$

```


APPENDIX C (BOOT.bin Generation)

```
nithingowtham25@zach-333-em4-05:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux$ petalin
ux-package --boot --fsbl zynq_fsbl.elf --fpga system.bit --u-boot u-boot.elf
[INFO] Sourcing buildtools
INFO: File in BOOT BIN: "/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux/zynq_fsbl.elf"
INFO: File in BOOT BIN: "/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux/system.bit"
INFO: File in BOOT BIN: "/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux/u-boot.elf"
INFO: File in BOOT BIN: "/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux/system.dtb"
INFO: Generating zynq binary package BOOT.BIN...

***** Bootgen v2023.1
**** Build date : Apr  7 2023-10:18:04
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.
** Copyright 2022-2023 Advanced Micro Devices, Inc. All Rights Reserved.

[WARNING]: Partition zynq_fsbl.elf.0 range is overlapped with partition system.bit.0 memory range
[WARNING]: Partition system.bit.0 range is overlapped with partition system.dtb.0 memory range

[INFO] : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
nithingowtham25@zach-333-em4-05:/tmp/petalinux_nithingowtham25/linux_boot_nithingowtham25/images/linux$
```

APPENDIX D (Linux Successful Bootup)

```
Mar 5 11:54
nithingowtham25@zach-333-em4-05:~/Spring_2026/ECEN749
nithingowtham25@zach-... x nithingowtham25@zach-... x nithingowtham25@zach-... x nithingowtham25@zach-... x
NET: Registered PF_PACKET protocol family
can: controller area network core
NET: Registered PF_CAN protocol family
can: raw protocol
mmc0: SDHCI controller on e0100000.mmc [e0100000.mmc] using ADMA
can: broadcast manager protocol
can: netlink gateway - max_hops=1
Registering SWP/SWPB emulation handler
of-fpga-region fpga-full: FPGA Region probed
of_cfs_init
of_cfs_init: OK
ALSA device list:
  No soundcards found.
mmc0: new high speed SDHC card at address 0001
mmcbldk0: mmc0:0001 MS 7.32 GiB
  mmcbldk0: p1
Freeing initrd memory: 12852K
Freeing unused kernel image (initmem) memory: 1024K
Run /init as init process
INIT: version 3.04 booting
Starting udev
Starting version 251.8+
ext3: Unknown parameter 'umask'
ext2: Unknown parameter 'umask'
ext4: Unknown parameter 'umask'
hwclock: can't open '/dev/misc/rtc': No such file or directory
Fri Mar  9 12:34:56 UTC 2018
hwclock: can't open '/dev/misc/rtc': No such file or directory
random: crng init done
Configuring packages on first boot....
(This may take several minutes. Please do not power off the machine.)
Running postinst /etc/rpm-postinsts/100-sysvinit-inittab...
update-rc.d: /etc/init.d/run-postinsts exists during rc.d purge (continuing)
Removing any system startup links for run-postinsts ...
  /etc/rcS.d/S99run-postinsts
INIT: Entering runlevel: 5
Configuring network interfaces... Cannot find device "eth0"
Starting OpenBSD Secure Shell server: sshd
  generating ssh RSA host key...
  generating ssh ECDSA host key...
  generating ssh ED25519 host key...
done.
Starting rpcbind daemon...done.
starting statd: done
hwclock: can't open '/dev/misc/rtc': No such file or directory
Starting internet superserver: inetd.
NFS daemon support not enabled in kernel
Starting syslogd/klogd: done
Starting tcf-agent: OK

Petalinux 2023.1+release-S05010539 linux_boot_nithingowtham25 ttyPS0
linux_boot_nithingowtham25 login: 
```


APPENDIX E (Entering Root Mode)

We trust you have received the usual lecture from the local System Administrator. It usually boils down to these three things:

- #1) Respect the privacy of others.
- #2) Think before you type.
- #3) With great power comes great responsibility.

Password:

linux_boot_nithingowtham25:/# cd /

linux_boot_nithingowtham25:/# ls -l

drwxr-xr-x	2	root	root	0	Mar	9	12:34	bin
drwxr-xr-x	2	root	root	0	Mar	9	12:34	boot
drwxr-xr-x	8	root	root	2560	Mar	9	12:35	dev
drwxr-xr-x	28	root	root	0	Mar	9	12:38	etc
drwxr-xr-x	4	root	root	0	Mar	9	12:34	home
lrwxrwxrwx	1	root	root	10	Mar	5	2026	init -> /sbin/init
drwxr-xr-x	7	root	root	0	Mar	9	12:34	lib
drwxr-xr-x	2	root	root	0	Mar	9	12:34	media
drwxr-xr-x	2	root	root	0	Mar	9	12:34	mnt
dr-xr-xr-x	106	root	root	0	Jan	1	1970	proc
drwx-----	2	root	root	0	Apr	21	2023	root
drwxr-xr-x	9	root	root	480	Mar	9	12:42	run
drwxr-xr-x	2	root	root	0	Mar	9	12:34	sbin
dr-xr-xr-x	12	root	root	0	Jan	1	1970	sys
lrwxrwxrwx	1	root	root	8	Mar	9	12:34	tmp -> /var/tmp
drwxr-xr-x	10	root	root	0	Mar	9	12:34	usr
drwxr-xr-x	8	root	root	0	Mar	9	12:34	var

linux_boot_nithingowtham25:/#